Taller 2 Corte: Investigación sobre React

Estudiante:	Sebastián Andrés Sará Rengifo
Código:	T00070996
Materia:	Desarrollo Front-end
Institución:	Universidad Tecnológica de Bolívar (UTB)
Modalidad:	Individual
Fecha de entrega:	Viernes 16 de octubre de 2026

Punto 1. ¿Qué es React?

¿Qué es y para qué sirve?

React es una herramienta de JavaScript que sirve para construir interfaces de usuario, es decir, la parte visible e interactiva de una página o aplicación web. Su idea central es dividir la pantalla en piezas pequeñas e independientes llamadas componentes (un botón, una tarjeta, un menú) que se combinan para formar páginas completas. Sirve para crear aplicaciones web dinámicas, como redes sociales o plataformas de video, donde la pantalla cambia constantemente sin recargar toda la página.

¿Quién lo creó y cuándo apareció?

React fue creado por Jordan Walke, ingeniero de Facebook (hoy Meta). Comenzó como un prototipo interno hacia 2011, se usó primero en funciones de Facebook como el «Me gusta» y los comentarios, y fue publicado como proyecto de código abierto en mayo de 2013 durante la conferencia JSConf US [3][4].

¿Librería o framework?

React es una **librería**. La diferencia está en quién controla la estructura del proyecto: un *framework* (como Angular) trae una forma completa y obligatoria de organizar la aplicación, incluyendo enrutamiento, manejo de datos y estructura de carpetas; una *librería* es una caja de herramientas enfocada en una tarea, y es el programador quien decide cómo integrarla. React se encarga únicamente de la capa visual (dibujar la interfaz según los datos). Para el enrutamiento, la obtención de datos o el despliegue hay que escoger otras herramientas, o usar frameworks construidos sobre React como Next.js [1]. Por eso lo considero una librería: ofrece libertad, pero a cambio exige tomar más decisiones.

¿Qué significa que React sea declarativo?

Significa que uno describe **cómo debe verse la interfaz** para un estado determinado de los datos, y no los pasos para modificarla. El programador dice «si hay 3 mensajes, muestra 3 mensajes», y React se encarga de actualizar la pantalla cuando los datos cambian. Es como pedir un plato en un restaurante (declaras lo que quieres) en lugar de entrar a la cocina a prepararlo paso a paso

(imperativo).

Punto 2. ¿Qué problema resuelve React?

La dificultad de construir interfaces «a mano»

Con JavaScript puro hay que manipular el DOM (el árbol de elementos de la página) directamente: buscar cada elemento, cambiar su texto, agregar o quitar nodos y escuchar eventos. En aplicaciones pequeñas esto es manejable, pero cuando la interfaz crece y muchos datos cambian al mismo tiempo, el código se vuelve largo, difícil de mantener y propenso a errores, porque es fácil que lo que se ve en pantalla quede desincronizado de los datos reales. Este era justamente el tipo de problema que enfrentaba Facebook al crecer [4].

Enfoque imperativo vs. declarativo

En el enfoque **imperativo** se escriben las instrucciones paso a paso. En el **declarativo** se describe el resultado y la librería resuelve los pasos. Ejemplo de un contador:

Imperativo (JavaScript puro):

```
const boton = document.getElementById("sumar");
const texto = document.getElementById("numero");
let contador = 0;

boton.addEventListener("click", () => {
  contador = contador + 1;
  texto.textContent = contador;   // yo debo actualizar la pantalla manualmente
});
```

Declarativo (React):

```
function Contador() {
  const [contador, setContador] = useState(0);

  return (
    <button onClick={() => setContador(contador + 1)}>
      Veces: {contador}
    </button>
  );
}
```

En el primer caso yo ordeno cada cambio en el DOM. En el segundo solo indico que el botón muestra el valor de `contador` y React actualiza la pantalla por mí cada vez que ese valor cambia.

Interfaz sincronizada con el estado

En React la interfaz es una función del estado: *UI = f(estado)*. Cuando el estado cambia, React vuelve a calcular cómo debería verse la pantalla, la compara con la versión anterior (mediante una copia liviana del DOM llamada Virtual DOM) y aplica en el DOM real solo las diferencias necesarias. Así los datos y lo que ve el usuario nunca quedan desincronizados, y el programador no tiene que rastrear cada cambio.

Rol de los componentes reutilizables

Dividir la interfaz en componentes permite escribir una pieza una sola vez y usarla muchas veces con datos distintos (por ejemplo, una tarjeta de producto repetida 50 veces). Esto reduce código duplicado, hace más fácil encontrar y corregir errores, y permite que varias personas trabajen en partes distintas de la aplicación sin estorbarse.

Punto 3. Principales componentes de React

3.1 Componentes

Definición: un componente es una pieza de interfaz reutilizable e independiente. En React moderno se crea como una función de JavaScript que devuelve lo que se debe mostrar en pantalla.

- Su nombre empieza con mayúscula (`Saludo`, no `saludo`).
- Se pueden anidar: un componente puede contener a otros.
- Cada uno maneja su propia lógica y su propia parte de la pantalla.

```
function Saludo() {
  return <h1>Hola, bienvenido a mi página</h1>;
}

function App() {
  return (
    <div>
      <Saludo />
      <Saludo />
    </div>
  );
}
```

Explicación: `Saludo` es un componente que devuelve un título. App lo usa dos veces, como si fuera una etiqueta HTML propia.

3.2 JSX

Definición: JSX es una extensión de la sintaxis de JavaScript que permite escribir código parecido a HTML dentro del mismo archivo de JavaScript. No lo entiende el navegador directamente: una herramienta (como Babel) lo convierte en llamadas normales de JavaScript.

- Un componente debe devolver un solo elemento raíz (se puede envolver en un `<div>` o en un fragmento `<>...</>`).
- Todas las etiquetas deben cerrarse, incluso las vacías: `<img />`.
- Se usa `className` en lugar de `class`, y los atributos van en camelCase (`onClick`).
- Con llaves `{ }` se inserta código JavaScript dentro del marcado.

```
function Perfil() {
  const nombre = "Sebastián";
  return (
    <div className="perfil">
      <h2>Hola, {nombre}</h2>
      <p>Resultado de 2 + 3: {2 + 3}</p>
```

```
    </div>
  );
}
```

Explicación: las llaves permiten mostrar la variable `nombre` y el resultado de una operación dentro del contenido, y se usa `className` para asignar una clase CSS.

3.3 Props

Definición: las props (de *properties*) son los datos que un componente padre le envía a un componente hijo. Funcionan como los parámetros de una función y sirven para personalizar un mismo componente con información diferente.

- Son de solo lectura: el componente que las recibe no debe modificarlas.
- Fluyen en una sola dirección: del padre al hijo.
- Pueden ser textos, números, listas, objetos o incluso funciones.

```
function Tarjeta({ nombre, carrera }) {
  return <p>{nombre} estudia {carrera}</p>;
}

function App() {
  return (
    <div>
      <Tarjeta nombre="Ana" carrera="Ingeniería de Sistemas" />
      <Tarjeta nombre="Luis" carrera="Diseño" />
    </div>
  );
}
```

Explicación: el mismo componente `Tarjeta` muestra información distinta según las props que recibe.

3.4 Estado (useState)

Definición: el estado es la memoria de un componente: datos que pueden cambiar con el tiempo (un contador, el texto de un formulario, si un menú está abierto). Se maneja con el *hook* `useState`, que devuelve el valor actual y una función para cambiarlo.

- Cuando el estado cambia mediante su función (por ejemplo `setActivo`), React vuelve a dibujar el componente automáticamente.
- No se debe modificar directamente (`activo = true` no funciona); siempre se usa la función asociada.
- A diferencia de las props, el estado pertenece al propio componente.

```
import { useState } from "react";

function Interruptor() {
  const [activo, setActivo] = useState(false);

  return (
    <button onClick={() => setActivo(!activo)}>
      {activo ? "Encendido" : "Apagado"}
    </button>
```

```
  );
}
```

Explicación: el estado `activo` empieza en `false`. Al presionar el botón se invierte su valor y React actualiza el texto mostrado («Encendido» o «Apagado») sin que yo toque el DOM.

3.5 Eventos (onClick, onChange)

Definición: los eventos son las acciones del usuario sobre la interfaz (hacer clic, escribir, enviar un formulario). En React se asocian a los elementos mediante atributos como `onClick` (al hacer clic) y `onChange` (cuando cambia el valor de un campo).

- Se les pasa una función (manejador), no un texto: `onClick={guardar}`.
- Se escriben en camelCase, a diferencia del HTML tradicional (`onclick`).
- Normalmente actualizan el estado, y así la interfaz cambia en respuesta al usuario.

```
function Formulario() {
  const [texto, setTexto] = useState("");

  return (
    <div>
      <input
        value={texto}
        onChange={(e) => setTexto(e.target.value)}
      />
      <button onClick={() => alert("Escribiste: " + texto)}>
        Enviar
      </button>
      <p>Vista previa: {texto}</p>
    </div>
  );
}
```

Explicación: cada vez que el usuario escribe, `onChange` guarda el texto en el estado y la vista previa se actualiza al instante; al hacer clic, `onClick` muestra un mensaje con lo escrito.

Punto 4. Importancia de React en la industria

¿Por qué tantas empresas lo eligen?

Principalmente porque ayuda a manejar interfaces grandes y cambiantes sin que el código se vuelva inmanejable. Los componentes reutilizables aceleran el desarrollo y mantienen una apariencia consistente; el enfoque declarativo hace el código más predecible; y es relativamente fácil de adoptar poco a poco en proyectos que ya existen. Además, el mismo conocimiento sirve para web y, con React Native, para aplicaciones móviles [4][5].

Aporte al ecosistema

Alrededor de React existen muchas herramientas: empaquetadores y servidores de desarrollo como Vite o webpack, frameworks construidos sobre React como Next.js o Remix, librerías de enrutamiento, manejo de estado, componentes visuales y pruebas. La propia documentación oficial

recomienda empezar proyectos nuevos con uno de estos frameworks [1]. Este ecosistema hace que casi cualquier necesidad ya tenga una solución probada.

Comunidad y mantenimiento

React es de código abierto desde 2013 y lo mantiene un equipo de ingenieros de Meta junto con colaboradores de la comunidad [2][3]. Su crecimiento y características posteriores, como los Hooks, se desarrollaron con la retroalimentación de la comunidad global. Una comunidad grande significa más tutoriales, más respuestas a dudas, más librerías y menos riesgo de que la tecnología quede abandonada.

Relevancia para un desarrollador front-end hoy

Aprender React es una habilidad muy valorada porque muchas empresas lo usan y, por tanto, muchas ofertas laborales lo piden. Además, sus conceptos (componentes, estado, flujo de datos declarativo) aparecen en otras tecnologías modernas, así que dominarlos facilita aprender otras herramientas del front-end.

Punto 5. Grandes empresas que utilizan React

1) Facebook (Meta)

Producto o sección: React nació dentro de Facebook y fue utilizado en funciones del sitio como el «Me gusta» y los comentarios, además del producto de publicidad (Ads) [4]. **Por qué es un caso relevante:** es el origen de la tecnología; la creación de React respondió a los problemas de escalar una interfaz enorme con muchas partes cambiando a la vez, lo que demuestra para qué fue diseñado.

2) Netflix

Producto o sección: el sitio web Netflix.com. El equipo reemplazó su generador de páginas basado en Java por uno en JavaScript que puede ejecutarse tanto en el servidor como en el navegador, y eligió React para lograrlo. Antes de decidir, construyeron dos prototipos competidores (uno solo en el cliente con Backbone y jQuery, y otro con JavaScript universal y React), y tras tres semanas la propuesta con React resultó la ganadora [6]. **Por qué es un caso relevante:** muestra una decisión técnica comparada y justificada para mejorar el tiempo de carga que perciben los usuarios.

3) Instagram

Producto o sección: el sitio web de Instagram. Tras ser adquirida por Facebook, Instagram se convirtió en el primer producto externo en usar React, luego de que Pete Hunt separara React de la infraestructura interna de Facebook para que pudiera usarse en otros proyectos [4]. **Por qué es un caso relevante:** fue la prueba de que React podía funcionar fuera de Facebook, un paso previo a su publicación como código abierto.

Fuentes consultadas

[1] React. *Documentación oficial de React*. https://react.dev (versión en español: https://es.react.dev)

[2] React. *Equipo y comunidad de React*. https://react.dev/community/team

[3] Gurubase. *The History of React*. https://gurubase.io/g/react/history-of-react

[4] Pusher. *The Evolution of React*. https://blog.pusher.com/the-evolution-of-react

[5] DEV Community. *The Untold Story of React*. https://dev.to/visy_ani/the-untold-story-of-react-4bja

[6] InfoQ (2015). *Netflix.com Adopts Universal JavaScript, Drops Java from Rendering Pipeline*. https://www.infoq.com/news/2015/08/netflix-universal-javascript/

[7] Meta Engineering Blog. https://engineering.fb.com